

Product Requirements Document

Autonomous AI Bug Hunter & Security Auditor

Document Owner: Product Management **Status:** Draft v1.0 **Last Updated:** August 29, 2026

1. Executive Summary

The Autonomous AI Bug Hunter & Security Auditor ("Hunter") is an AI-driven system that continuously and autonomously discovers, verifies, triages, and reports security vulnerabilities and functional bugs across an organization's codebase, APIs, and running infrastructure. It combines static analysis, dynamic testing, dependency auditing, and LLM-driven reasoning to behave like a tireless senior security engineer — reading code, forming hypotheses about weaknesses, writing and executing proof-of-concept exploits in sandboxed environments, and producing developer-ready fixes with evidence.

Unlike traditional SAST/DAST tools that generate high volumes of low-confidence alerts, Hunter is designed to close the loop: **find → verify → explain → propose fix → track remediation**, with human review gates at points where autonomous action carries risk.

2. Problem Statement

2.1 Current Pain Points

Problem	Impact
Security teams are outnumbered by developers and code volume	Vulnerabilities ship to production before review
Traditional SAST/DAST tools produce 60–90% false positives	Alert fatigue causes real issues to be ignored
Manual penetration testing happens 1–4 times/year	Long windows of undetected exposure between tests
Bug bounty programs are reactive and expensive	Critical issues found only after external actors look
Fix guidance from scanners is generic	Developers spend more time triaging than fixing
Dependency and supply-chain risks change daily	Point-in-time audits go stale immediately

2.2 Opportunity

Recent advances in LLM code reasoning, agentic tool use, and sandboxed execution make it feasible to build a system that doesn't just pattern-match known vulnerability signatures, but reasons about code the way a human auditor does — understanding business logic, tracing data flow across services, and constructing real exploit chains. This shifts security testing from periodic and sample-based to continuous and comprehensive.

3. Goals & Success Metrics

3.1 Product Goals

Continuously discover exploitable vulnerabilities (not just style/lint issues) with high precision.
Autonomously verify findings via safe, sandboxed proof-of-concept execution before reporting.
Reduce mean-time-to-remediation (MTTR) by providing verified, actionable, developer-ready fixes.
Operate safely within defined guardrails — never causing production impact or unauthorized access.
Integrate seamlessly into existing SDLC (CI/CD, ticketing, IDE) so findings reach developers where they work.

3.2 Success Metrics (KPIs)

Metric	Baseline (industry avg.)	Target at GA
False positive rate	60–90% (traditional SAST)	< 10%
Mean time to detect (MTTD) critical vuln	Weeks–months	< 24 hours
Mean time to remediate (MTTR), critical	30–90 days	< 7 days
% findings with verified PoC	~0% (most tools)	> 95% for exploitable classes
Developer fix-acceptance rate (suggested patch)	N/A	> 60%
Coverage: repos/services actively monitored	N/A	100% of in-scope assets
Escaped vulnerabilities (found externally first)	N/A	Reduce by 80% YoY

4. Target Users & Personas

1. Application Security Engineer ("Priya") Owns the security posture of 200+ services. Needs high-confidence findings, exploit evidence, and the ability to tune scope/rules without writing new detection logic from scratch.

2. Software Engineer / Tech Lead ("Marcus") Wants actionable, low-noise findings inline in PRs, with clear reproduction steps and suggested fixes — not a 40-page PDF report.

- 3. CISO / Security Leadership ("Elena")** Needs organization-wide risk visibility, trend reporting, compliance mapping (SOC 2, ISO 27001, PCI-DSS), and audit trails of what was found, verified, and fixed.
- 4. DevOps / Platform Engineer ("Sam")** Responsible for CI/CD pipeline health and infrastructure. Needs Hunter to run without degrading build times or violating infra access boundaries.
- 5. Compliance / Risk Officer ("Dana")** Needs exportable evidence for audits and regulatory reporting, and assurance that autonomous testing itself is governed and logged.

5. Scope

5.1 In Scope (v1 / GA)

Static analysis (SAST+): LLM-augmented code review across common languages (Python, JavaScript/TypeScript, Java, Go, C#, Ruby) for vulnerability classes: injection (SQL/NoSQL/command/LDAP), XSS, SSRF, IDOR/broken access control, authN/authZ flaws, insecure deserialization, path traversal, race conditions, secrets in code, insecure crypto usage, business-logic flaws.

Dynamic analysis (DAST): Autonomous black-box and grey-box testing of staging/pre-prod web apps and APIs (REST, GraphQL) via sandboxed agents.

Dependency & SCA auditing: Continuous scanning of open-source dependencies, transitive risk, license issues, and known-CVE correlation with reachability analysis (is the vulnerable code path actually reachable?).

Secrets detection: Repo, commit-history, and runtime config scanning for exposed credentials.

IaC & config auditing: Terraform, CloudFormation, Kubernetes manifests, Dockerfiles — misconfiguration detection (e.g., open S3 buckets, overly permissive IAM).

Exploit verification sandbox: Isolated, network-segmented environments where Hunter reproduces an exploit safely before reporting it as "confirmed."

Fix generation: AI-generated patch suggestions with explanation, submitted as draft PRs (not auto-merged).

Reporting & integration: Jira/Linear/GitHub Issues ticketing, Slack/Teams alerts, SARIF export, PR/MR checks, executive dashboards.

Guardrails & governance: Scope enforcement, rate limiting, kill switch, full audit logging of every autonomous action.

5.2 Out of Scope (v1)

Fully autonomous production penetration testing (production testing requires explicit human-approved authorization windows).

Physical security, social engineering, or phishing simulation.

Autonomous patch auto-merge without human approval (roadmap item, opt-in only, post-GA).

Mobile binary reverse engineering (iOS/Android) — planned for v2.

Hardware/firmware/embedded systems security.

5.3 Explicitly Prohibited Behaviors

No scanning or exploitation of assets outside a defined, authorized scope/allowlist.

No autonomous action against third-party systems not owned by the customer.

No credential harvesting, data exfiltration, or lateral movement beyond what's needed to verify a specific finding in an isolated sandbox.

No autonomous production changes.

6. Product Requirements

6.1 Discovery Engine

ID	Requirement	Priority
DE-1	System shall ingest source code from Git-based repos (GitHub, GitLab, Bitbucket) via read-only integration.	P0
DE-2	System shall build a semantic map of the application (data flow graph, entry points, trust boundaries, auth flows) rather than relying solely on regex/AST pattern rules.	P0
DE-3	System shall reason across multi-file and multi-service boundaries (e.g., trace a tainted input from an API gateway through microservices to a database call).	P0
DE-4	System shall re-scan incrementally on every PR/commit (diff-aware) and perform full deep scans on a configurable schedule (e.g., nightly/weekly).	P0
DE-5	System shall prioritize findings using a risk score combining exploitability, data sensitivity, exposure (internet-facing vs internal), and business context.	P0
DE-6	System shall support custom detection rules/policies defined by security teams (e.g., org-specific insecure patterns).	P1
DE-7	System shall detect and flag AI/LLM-specific vulnerabilities in customer applications that use LLMs (prompt injection, insecure tool use, excessive agency).	P1

6.2 Verification & Exploitation Sandbox

ID	Requirement	Priority
VS-1	Every finding classified as "exploitable" must be verified via automated PoC execution in an isolated, network-segmented sandbox before being marked "confirmed."	P0
VS-2	Sandbox environments shall be ephemeral, destroyed after each verification run, and never share network access with production.	P0

ID	Requirement	Priority
VS-3	System shall clearly label unverified findings as "potential" vs. verified findings as "confirmed," with the PoC artifact attached.	P0
VS-4	System shall impose hard resource, time, and rate limits on all sandbox/dynamic testing activity to prevent DoS-like side effects.	P0
VS-5	All exploit code generated shall be retained in an auditable, access-controlled evidence store — never executed outside the sandbox boundary.	P0
VS-6	Any test that could plausibly affect a live, non-sandboxed system shall require explicit human pre-approval and a defined authorization window.	P0

6.3 Triage & Reporting

ID	Requirement	Priority
TR-1	Each finding includes: severity (CVSS-aligned), affected asset, data flow trace, verified PoC/reproduction steps, business impact narrative, and suggested remediation.	P0
TR-2	System shall deduplicate findings across scan cycles and correlate recurring issues to a single tracked entity.	P0
TR-3	System shall auto-create tickets in the org's issue tracker with severity-based SLAs and auto-assign based on code ownership (CODEOWNERS mapping).	P0
TR-4	System shall generate a draft fix as a pull/merge request, never auto-merged, always requiring human review and CI pass.	P0
TR-5	System shall provide an executive/compliance dashboard: risk trends, open vs. resolved, coverage %, mapped to frameworks (OWASP Top 10, CWE, SOC 2, PCI-DSS).	P1
TR-6	System shall support suppression/false-positive marking with feedback loop to improve future precision.	P1
TR-7	System shall generate exportable audit reports (PDF/SARIF/CSV) for compliance and third-party auditors.	P1

6.4 Governance, Safety & Access Control

ID	Requirement	Priority
GV-1	All scanning/testing scope must be explicitly allowlisted by the customer (repos, domains, IP ranges); nothing outside scope is touched.	P0

ID	Requirement	Priority
GV-2	System shall provide a global and per-project "kill switch" to immediately halt all autonomous activity.	P0
GV-3	Every autonomous action (scan started, exploit attempted, PR opened, ticket created) shall be logged immutably with timestamp, actor (agent/human), and justification.	P0
GV-4	Role-based access control: separate permissions for viewing findings, approving dynamic tests, approving production-adjacent testing windows, and managing scope.	P0
GV-5	System shall operate under least-privilege, read-only credentials wherever possible; any elevated/write access must be time-boxed and logged.	P0
GV-6	Data handling: source code and findings shall be encrypted at rest and in transit; customer data shall not be used to train shared models without explicit opt-in.	P0
GV-7	System shall support on-prem/VPC-isolated deployment for customers with strict data residency requirements.	P1

6.5 Integrations

ID	Requirement	Priority
IN-1	Git providers: GitHub, GitLab, Bitbucket (Cloud + self-hosted).	P0
IN-2	CI/CD: GitHub Actions, GitLab CI, Jenkins, CircleCI — as a pipeline gate/status check.	P0
IN-3	Ticketing: Jira, Linear, GitHub Issues, Azure DevOps.	P0
IN-4	Chat/alerting: Slack, Microsoft Teams.	P0
IN-5	SIEM/SOAR export: Splunk, Datadog, generic webhook/SARIF.	P1
IN-6	IDE plugin (VS Code, JetBrains) for inline findings during development.	P2
IN-7	Cloud provider integration (AWS, GCP, Azure) for IaC and live config auditing (read-only).	P1

7. Non-Functional Requirements

Category	Requirement
Performance	Incremental PR scans complete in < 5 minutes for typical diffs; full deep scans do not block deploys.

Category	Requirement
Scalability	Support orgs with 10,000+ repositories and multi-million-line monorepos.
Reliability	99.9% uptime for the control plane; scan queue must gracefully degrade under load (no dropped findings).
Security	The product itself must pass independent third-party penetration testing before GA; SOC 2 Type II required at launch.
Auditability	100% of autonomous actions must be reconstructable from logs for at least 1 year (configurable retention).
Explainability	Every finding must include a human-readable reasoning trace, not just a verdict.
Cost control	LLM/compute usage per scan shall be bounded and predictable; customers can set spend caps per project.

8. User Experience & Workflows

8.1 Core Workflow: PR-Triggered Scan

Developer opens a pull request.
 Hunter performs an incremental diff-aware scan within the CI pipeline.
 If a high-confidence, verified finding is detected, Hunter posts an inline PR comment with severity, explanation, and a proposed fix diff.
 PR status check fails for Critical/High severity confirmed findings (configurable policy), passes with a warning for Medium/Low.
 Developer accepts the suggested fix, requests more detail, or marks as a false positive (with required justification, logged for tuning).

8.2 Core Workflow: Scheduled Deep Audit

Security team defines scope (repos, staging environments, API specs) and a testing window.
 Hunter runs a full static + dynamic audit, including sandboxed exploit verification.
 Findings are triaged, deduplicated, and pushed to the ticketing system with SLA-based due dates.
 Weekly digest and live dashboard update for the security team; monthly executive summary for leadership.

8.3 Core Workflow: Incident-Triggered Deep Dive

A new CVE is published affecting a dependency in use.
 Hunter performs reachability analysis: is the vulnerable function actually called, and is it reachable from an untrusted input?
 If reachable, Hunter escalates immediately (out-of-band alert) with exploitability assessment and suggested upgrade/patch path, bypassing the normal scan cadence.

9. Risks & Mitigations

Risk	Likelihood Impact		Mitigation
Autonomous exploit testing causes unintended production impact	Medium	Critical	Strict sandbox isolation, scope allowlisting, kill switch, no production testing without explicit human authorization window
False positives erode developer trust	Medium	High	Mandatory PoC verification before "confirmed" label; feedback loop; precision-focused tuning over recall
LLM hallucinated vulnerabilities or fixes	Medium	High	All findings require verification step; all fixes require human review/CI pass before merge; no auto-merge
Customer source code / IP exposure	Low	Critical	Encryption at rest/in transit, no shared-model training without opt-in, on-prem/VPC deployment option, SOC 2 Type II
Malicious use of generated exploit code	Low	High	Exploit artifacts stored in access-controlled evidence vault, never exported raw, watermarked/logged access
Regulatory/compliance concerns around autonomous testing	Medium	Medium	Built-in authorization workflows, immutable audit logs, compliance mapping and reporting features
Alert fatigue if precision targets are missed	Medium	High	Phased rollout starting in "shadow/advisory mode" before enforcing PR-blocking gates

10. Rollout Plan

Phase 0 — Shadow Mode (Beta): Hunter runs read-only, posts findings to a private channel/dashboard only. No PR blocking, no autonomous dynamic testing. Goal: measure precision, tune thresholds.

Phase 1 — Assisted Mode (Limited GA): PR comments and draft-fix PRs enabled. Dynamic/exploit verification limited to explicitly approved staging environments. Human approval required for all dynamic tests.

Phase 2 — Guarded Autonomy (GA): Diff-aware scanning fully automated in CI. Scheduled deep audits with sandboxed verification run autonomously within pre-approved scope and time windows. PR-blocking gates enabled for Critical/High confirmed findings (opt-in policy).

Phase 3 — Expanded Autonomy (Post-GA): Optional auto-merge for low-risk, high-confidence fixes (e.g., dependency version bumps) under strict customer-defined policy. Mobile and IaC-runtime coverage expansion.

11. Open Questions

What is the liability/indemnification model if Hunter's autonomous testing causes downtime despite guardrails?

Should customers be able to bring their own LLM/inference endpoint for data-residency-sensitive environments?

What is the pricing model — per repository, per developer seat, per scan, or per finding verified?

How do we handle findings in third-party/vendor code that the customer cannot directly patch?

What's the escalation path when Hunter and a human reviewer disagree on severity or exploitability?

12. Appendix: Glossary

SAST — Static Application Security Testing

DAST — Dynamic Application Security Testing

SCA — Software Composition Analysis

PoC — Proof of Concept (exploit)

IDOR — Insecure Direct Object Reference

SSRF — Server-Side Request Forgery

IaC — Infrastructure as Code

MTTR / MTTD — Mean Time to Remediate / Detect

SARIF — Static Analysis Results Interchange Format